

Lab1 实验报告

姓名：王思博 学号：25300190047

仓库地址：<https://github.com/sibo25/Lab1>

1. signMask: 生成最高位掩码

题目要求：

返回一个仅最高位为 1、其余位均为 0 的 32 位整数，即 0x80000000。最多允许使用 2 次运算。

实现思路：

整数 1 的二进制表示中，只有最低位为 1。将其左移 31 位，即可把该位移动到最高位，低位补 0，得到目标掩码。

实现代码：

```
int signMask(void) {  
    return 1 << 31;  
}
```

测试结果：

该实现使用了 1 次左移运算，未超过 2 次运算的限制。通过 btest 的单题测试，得分为 1/1；操作合法性检查显示为 1/2 operators。

2. bitXor: 使用按位与和按位取反实现异或

题目要求：

实现 $x \oplus y$ 的功能，仅允许使用按位与 $\&$ 和按位取反 $\sim$ ，最多允许使用 8 次运算。

实现思路：

异或运算对每个二进制位分别进行判断：两个对应位不同时，结果为 1；相同时，结果为 0。

两个对应位相同的情况包括“均为 1”和“均为 0”。其中， $x \& y$ 标记两个对应位均为 1 的位置， $\sim x \& \sim y$ 标记两个对应位均为 0 的位置。分别对这两个结果取反，再进行按位与，就能保留两个对应位不同的位置，实现异或运算。

实现代码：

```
int bitXor(int x, int y) {  
    return ~(x & y) & ~(~x & ~y);  
}
```

测试结果：

该实现使用了 4 次按位取反和 3 次按位与，共 7 次运算，未超过 8 次运算的限制。通过 btest 的单题测试，得分为 2/2；操作合法性检查显示为 7/8 operators。

3. negativePart：提取负数的相反数

题目要求：

当 x 为负数时，返回其相反数；否则返回 0。最多允许使用 6 次运算。

实现思路：

首先，将 x 算术右移 31 位，生成符号掩码。当 x 为负数时，掩码的所有位均为 1；当 x 为非负数时，所有位均为 0。

然后，利用补码的“取反加一”规则，通过 $\sim x + 1$ 计算 x 的相反数，再与符号掩码进行按位与。这样，负数对应的计算结果被保留，非负数对应的结果被置为 0。

实现代码：

```
int negativePart(int x) {  
    int mask = x >> 31;  
    return (~x + 1) & mask;  
}
```

测试结果：

该实现共使用 4 次运算，未超过 6 次运算的限制。通过 btest 的单题测试，得分为 3/3，错误数为 0；操作合法性检查显示为 4/6 operators。

4. copyByteWithin：整数内部的字节复制

题目要求：

将整数 x 中编号为 src 的字节复制到编号为 dst 的字节，其余字节保持不变。字节从最低位开始编号为 $0\sim3$ ，最多允许使用 12 次运算。

实现思路：

首先，将 x 右移 $src \ll 3$ 位，并与 $0xFF$ 按位与，提取源字节。随后，将源字节左移 $dst \ll 3$ 位，使其到达目标位置。

为了清空原有的目标字节，将 $0xFF$ 左移到目标位置后取反，再与 x 按位与。最后，将清空后的整数与移位后的源字节合并，完成复制。

由于每个字节包含 8 位，字节编号左移 3 位即可得到对应的位移量。当 src 与 dst 相同时，这一过程会将原字节填回原位置，结果保持不变。

实验代码：

```
int copyByteWithin(int x, int src, int dst) {
    return (((x >> (src << 3)) & 0xFF) << (dst << 3))
        | (x & ~(0xFF << (dst << 3)));
}
```

测试结果

该实现共使用 10 次运算，未超过 12 次运算的限制。通过 btest 单题测试，得分为 4/4，错误数为 0；操作合法性检查显示为 10/12 operators。

5. logicalShift: 实现逻辑右移

题目要求：

将整数 x 逻辑右移 n 位，左侧空出的位全部补 0。其中 n 的范围为 $0\sim31$ ，最多允许使用 20 次运算。

实现思路：

实验中的 int 右移为算术右移，负数右移时会在高位补 1。因此，先计算 $x \gg n$ ，再通过掩码将最高 n 位清零，即可实现逻辑右移。

构造掩码时，先通过 $1 \ll 31$ 得到仅最高位为 1 的整数，再算术右移 n 位，使最高 $n+1$ 位为 1。随后左移 1 位，得到最高 n 位为 1、其余位为 0 的值。对其取反，就得到最高 n 位为 0、其余位为 1 的掩码。

当 n 为 0 时，掩码为全 1，结果保持 x 不变；当 n 为 31 时，掩码仅最低位为 1，保留移位后的最低位。

实现代码：

```
int logicalShift(int x, int n) {
    int mask = ~(((1 << 31) >> n) << 1);
    return (x >> n) & mask;
}
```

测试结果

该实现共使用 6 次运算，未超过 20 次运算的限制。通过 btest 单题测试，得分为 4/4，错误数为 0；操作合法性检查显示为 6/20 operators。

6. swapNibblePairs: 交换每个字节的高低半字节

题目要求:

交换整数 x 的每个字节中的高 4 位和低 4 位，最多允许使用 18 次运算。例如，0x12345678 交换后得到 0x21436587。

实现思路:

首先，从常量 0x0F 出发，通过左移和按位或构造掩码 0x0F0F0F0F，用于提取每个字节的低 4 位。

将 x 与掩码按位与后左移 4 位，把各字节原来的低 4 位移动到高 4 位。另一方面，将 x 右移 4 位后与掩码按位与，提取各字节原来的高 4 位，使其位于低 4 位，同时清除算术右移补入的高位。

最后，将两部分按位或，完成每个字节内部的高低半字节交换。

实现代码:

```
int swapNibblePairs(int x) {
    int mask = 0x0F;
    mask = mask | (mask << 8);
    mask = mask | (mask << 16);

    return ((x & mask) << 4) | ((x >> 4) & mask);
}
```

测试结果:

该实现共使用 9 次运算，未超过 18 次运算的限制。通过 btest 单题测试，得分为 4/4，错误数为 0；操作合法性检查显示为 9/18 operators。

7. secondLowestZeroBit: 定位第二个最低的零位

题目要求:

从最低位向最高位查找整数 x 中第二个为 0 的位, 返回仅该位置为 1、其余位置均为 0 的掩码。如果不足两个零位, 则返回 0。最多允许使用 8 次运算。

实现思路:

首先对 x 按位取反, 得到 y , 将寻找第二个最低零位的问题转化为寻找第二个最低的 1。

然后利用 $y \& (y - 1)$ 清除 y 中最低的 1。由于实验不允许使用减号, 用 $y + \sim 0$ 表示 $y - 1$, 得到中间结果 z 。

最后, 通过 $z \& (\sim z + 1)$ 提取 z 中最低的 1, 即原来 x 中第二个最低零位对应的掩码。如果 x 中不足两个零位, 则 z 为 0, 最终结果也为 0。

实现代码:

```
int secondLowestZeroBit(int x) {  
    int y = ~x;  
    int z = y & (y + ~0);  
    return z & (~z + 1);  
}
```

测试结果:

该实现共使用 7 次运算, 未超过 8 次运算的限制。通过 btest 单题测试, 得分为 4/4, 错误数为 0; 操作合法性检查显示为 7/8 operators。

8. oddParity: 计算奇校验位

题目要求:

当整数 x 的 32 位二进制表示中包含偶数个 1 时返回 1, 包含奇数个 1 时返回 0, 使原数中的 1 与返回的校验位合计为奇数个。最多允许使用 56 次运算。

实现思路

利用异或运算判断 1 的数量的奇偶性: 所有位异或的结果在有奇数个 1 时为 1, 在有偶数个 1 时为 0。

依次将 x 与自身右移 16、8、4、2、1 位的结果进行异或, 逐步将全部 32 位的奇偶信息合并到最低位。每一步所需的低位仅依赖上一轮有效范围内的位, 因此无须在中间步骤清除高位。

最后，通过 $x \& 1$ 提取最低位，再使用逻辑非！翻转结果，得到题目要求的奇校验位。

实现代码：

```
int oddParity(int x) {  
    x = x ^ (x >> 16);  
    x = x ^ (x >> 8);  
    x = x ^ (x >> 4);  
    x = x ^ (x >> 2);  
    x = x ^ (x >> 1);  
  
    return !(x & 1);  
}
```

测试结果：

该实现共使用 12 次运算，未超过 56 次运算的限制。通过 btest 单题测试，得分为 5/5，错误数为 0；操作合法性检查显示为 12/56 operators。

9. rotateRightBits: 实现循环右移

题目要求

将整数 x 循环右移 n 位，右端移出的位从左端补入。 n 为非负整数，实际移位量按 32 取余。最多允许使用 16 次运算。

实现思路

首先使用 $n \& 31$ 得到实际移位量 k ，使其处于 $0 \sim 31$ 的范围内。

将循环右移拆成两部分：一部分将 x 算术右移 k 位，再用掩码清除高位补入的符号位，实现逻辑右移；另一部分将 x 左移，使原来的低 k 位移动到最高处。最后通过按位或合并两部分。

左移量使用 $(32 + (\sim k + 1)) \& 31$ 计算。当 k 非零时，左移量为 $32 - k$ ；当 k 为 0 时，左移量为 0，避免移位 32 位。此时两部分均为 x ，按位或后仍为 x 。

实现代码

```
int rotateRightBits(int x, int n) {  
    int k = n & 31;
```

```

    int mask = ~(((1 << 31) >> k) << 1);
    int right = (x >> k) & mask;
    int leftShift = (32 + (~k + 1)) & 31;

    return right | (x << leftShift);
}

```

测试结果

该实现共使用 13 次运算，未超过 16 次运算的限制。通过 btest 单题测试，得分为 5/5，错误数为 0；操作合法性检查显示为 13/16 operators。

10. roundEvenPow2: 舍入到最近的 2 的幂的倍数

题目要求

将非负整数 x 舍入到最近的 (2^n) 的整数倍。如果 x 恰好位于两个倍数的中间，则选择倍数系数为偶数的结果。最多允许使用 24 次运算。

实现思路

首先通过 $x \gg n$ 得到商 q ，并使用低 n 位全为 1 的掩码提取余数 r 。随后，将余数与 (2^{n-1}) 比较，确定是否需要向上舍入。

通过 $\text{half} - r$ 的符号判断余数是否大于一半；通过 $!(r \wedge \text{half})$ 判断余数是否恰好等于一半。当余数大于一半，或者余数等于一半且商为奇数时，将商加 1；其他情况下保持商不变。最后将调整后的商左移 n 位，得到舍入结果。

实现代码

```

int roundEvenPow2(int x, int n) {
    int q = x >> n;
    int mask = (1 << n) + ~0;
    int r = x & mask;
    int half = 1 << (n + ~0);
    int diff = half + (~r + 1);
    int greater = (diff >> 31) & 1;
    int tie = !(r ^ half);
    int up = greater | (tie & (q & 1));
}

```

```
    return (q + up) << n;
}
```

测试结果

通过 btest 单题测试，得分为 5/5，错误数为 0。上述代码共使用 20 次运算，未超过 24 次运算的限制。

11. midpointTowardFirst: 向第一个参数取整的中点

题目要求

计算整数 x 和 y 的中点。如果中点不是整数，则向第一个参数 x 的方向取整。计算需避免溢出造成错误，最多允许使用 32 次运算。

实现思路

首先分别将 x 和 y 算术右移一位，再相加。当两者均为奇数时，补回分别右移所舍去的两个二分之一，得到向下取整的中点。这样可以避免直接计算 $x + y$ 导致溢出。

随后，通过 $(x \wedge y) \& 1$ 判断两者奇偶性是否不同，即中点是否带有小数部分。判断 x 是否大于 y 时，分同号和异号处理：异号时直接根据 x 的符号判断；同号时根据 $y - x$ 的符号判断，避免相减溢出影响比较结果。

当中点带有小数部分且 x 大于 y 时，将向下取整的结果加 1；否则保持不变，从而实现向 x 的方向取整。

实现代码

```
int midpointTowardFirst(int x, int y) {
    int mid = (x >> 1) + (y >> 1) + (x & y & 1);
    int odd = (x ^ y) & 1;

    int sx = (x >> 31) & 1;
    int sy = (y >> 31) & 1;
    int different = sx ^ sy;
    int diff = y + (~x + 1);

    int greater = (different & !sx)
```

```

        | ((!different) & ((diff >> 31) & 1));

    return mid + (odd & greater);
}

```

测试结果：

该实现共使用 25 次运算，未超过 32 次运算的限制。通过 btest 单题测试，得分为 5/5，错误数为 0；操作合法性检查显示为 25/32 operators。

12. isBetweenEitherOrder：判断是否位于两个端点之间

题目要求：

判断整数 x 是否位于 a 和 b 构成的闭区间内，端点顺序不限。满足时返回 1，否则返回 0。最多允许使用 48 次运算。

实现思路：

分别判断 x 是否小于 a 、是否小于 b 。比较时区分同号和异号：异号时根据 x 的符号判断；同号时根据差值的符号判断，从而避免减法溢出导致错误。

当 x 不等于两个端点时，若两次比较结果不同，则说明 x 位于两个端点之间。因此，使用异或合并两次比较结果，再通过按位或加入 x 等于 a 或 b 的情况，实现包含端点且不依赖端点顺序的区间判断。

实现代码：

```

int isBetweenEitherOrder(int x, int a, int b) {
    int sx = (x >> 31) & 1;
    int sa = (a >> 31) & 1;
    int sb = (b >> 31) & 1;

    int differentA = sx ^ sa;
    int differentB = sx ^ sb;
    int diffA = x + (~a + 1);
    int diffB = x + (~b + 1);

    int lessA = (differentA & sx)

```

```

        | ((!differentA) & ((diffA >> 31) & 1));
int lessB = (differentB & sx)
        | ((!differentB) & ((diffB >> 31) & 1));

return (lessA ^ lessB) | !(x ^ a) | !(x ^ b);
}

```

测试结果：

通过 btest 单题测试，得分为 7/7，错误数为 0。上述代码共使用 33 次运算，未超过 48 次运算的限制。

13. mul5Sat: 实现乘以 5 的饱和运算

题目要求：

计算整数 x 乘以 5 的结果。如果数学结果超过 32 位有符号整数的上限，则返回最大整数；如果低于下限，则返回最小整数。最多允许使用 30 次运算。

实现思路：

通过三次加法依次计算 x 的 2 倍、4 倍和 5 倍，并利用异或检查各中间结果与 x 的符号是否不同。如果出现符号变化，则说明计算过程中发生了溢出。将检测结果算术右移 31 位，生成全 1 或全 0 的溢出掩码。

随后，根据 x 的符号确定饱和值： x 非负时取最大整数， x 为负时取最小整数。最后利用掩码选择返回值，溢出时返回饱和值，否则返回乘以 5 的计算结果。

实现代码：

```

int mul5Sat(int x) {
    int x2 = x + x;
    int x4 = x2 + x2;
    int x5 = x4 + x;

    int overflow = ((x ^ x2) | (x ^ x4) | (x ^ x5)) >> 31;
    int max = ~(1 << 31);
    int limit = max ^ (x >> 31);
}

```

```
    return (overflow & limit) | (~overflow & x5);  
}
```

测试结果:

通过 btest 单题测试, 得分为 7/7, 错误数为 0。上述代码共使用 17 次运算, 未超过 30 次运算的限制。

14. classifyAdd3: 判断三个整数之和是否越界

题目要求:

判断三个整数的数学和是否超出 32 位有符号整数范围: 超过上限返回 1, 低于下限返回 -1, 否则返回 0。最多允许使用 52 次运算。

实现思路:

将每个整数拆分为除以 4 后向下取整的商, 以及范围为 0~3 的余数。商通过算术右移 2 位获得, 余数通过与 3 按位与获得。因为 3 的二进制是 11, 与 3 按位与, 就只保留最低两位。而最低两位恰好表示除以 4 的余数。

分别累加三个商和三个余数, 再将余数和产生的进位加入商的总和, 得到 high。它等于三个整数的数学和除以 4 后向下取整的结果, 且计算过程不会溢出。

原总和在合法范围内, 等价于 high 位于 $[-2^{29}, 2^{29}-1]$ 。因此, 通过 $high - 2^{29}$ 和 $high + 2^{29}$ 的符号判断是否超过上限或低于下限, 最后合并得到分类结果。

实现代码:

```
int classifyAdd3(int x, int y, int z) {  
    int low = (x & 3) + (y & 3) + (z & 3);  
    int high = (x >> 2) + (y >> 2) + (z >> 2)  
               + (low >> 2);  
  
    int bound = 1 << 29;  
    int above = !((high + (~bound + 1)) >> 31);  
    int below = (high + bound) >> 31;
```

```
    return above | below;
}
```

测试结果：

通过 btest 单题测试，得分为 7/7，错误数为 0。上述代码共使用 21 次运算，未超过 52 次运算的限制。

15. floatScaleThreeHalves：计算浮点数的 1.5 倍

题目要求

使用整数运算，返回单精度浮点数乘以 1.5 后的二进制编码。结果采用就近舍入、平局取偶数的规则，并正确处理正负零、非规格化数、无穷和 NaN。最多允许使用 60 次运算。

实现思路

首先提取符号位、指数字段和小数字段。对于无穷和 NaN，直接返回原编码；对于规格化数，补回隐藏的最低位 1，得到完整的有效数字。

将有效数字乘以 3，再通过右移完成除以 2。如果乘法使规格化数的有效数字达到或超过 2，则增加指数，并额外右移一位，使其恢复规格化形式。

右移时保留被舍弃部分的信息：余数大于一半时进位，恰好等于一半时根据保留部分的最低位决定是否进位，实现平局取偶数。

最后处理非规格化数转为规格化数、舍入引起的进位以及指数溢出。指数溢出时返回同符号的无穷，否则将符号位、指数和小数字段重新组合。

实现代码：

```
unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFFFF;
    unsigned significand;
    unsigned product;
    unsigned shift = 1;
    unsigned result;
    unsigned remainder;
```

```

unsigned half;
if (exp == 255) {
    return uf;
}
significand = frac;
if (exp != 0) {
    significand = significand | 0x800000;
}
product = significand + (significand << 1);
if (exp != 0 && product >= 0x2000000) {
    shift = 2;
    exp = exp + 1;
}
result = product >> shift;
remainder = product & ((1 << shift) - 1);
half = 1 << (shift - 1);

if (remainder > half ||
    (remainder == half && (result & 1))) {
    result = result + 1;
}
if (exp == 0) {
    if (result >= 0x800000) {
        exp = 1;
    }
} else {
    if (result >= 0x1000000) {
        result = result >> 1;
        exp = exp + 1;
    }
}

```

```

    }
}
if (exp >= 255) {
    return sign | 0x7F800000;
}
return sign | (exp << 23) | (result & 0x7FFFFFFF);
}

```

测试结果

通过 btest 单题测试，得分为 7/7，错误数为 0。操作合法性及运算次数以 check_ops.py 的对应输出为准。

16. floatRoundEven: 浮点数的就近取整

题目要求

将参数 uf 表示的单精度浮点数舍入到最近的整数，并返回该结果的浮点数编码。当输入恰好位于两个整数的中点时，选择偶数。保留零的符号，对于 NaN 和无穷，返回原编码。

实现思路

首先提取符号位、阶码和尾数，根据阶码分情况处理：

1. 当阶码大于或等于 150 时，有限数的实际指数至少为 23，已经没有小数部分，直接返回原编码。NaN 和无穷也在此分支中原样返回。
2. 当阶码小于 126 时，数的绝对值小于 0.5，返回保留原符号的零。
3. 当阶码等于 126 时，数的绝对值位于区间 $[0.5, 1)$ 。若尾数为零，则恰好为 0.5，按“平局取偶数”舍入到零；否则舍入到对应符号的 1。
4. 对于其余情况，实际指数为 $e = \text{exp} - 127$ ，编码中对应小数部分的位数为 $23 - e$ 。利用掩码提取并清除这些位，再将被清除部分与半个单位比较：大于一半时进位，小于一半时不进位；恰好等于一半时，仅在整数部分为奇数时进位。

实现代码

```

unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000;

```

```

unsigned exp = (uf >> 23) & 0xFF;
unsigned frac = uf & 0x7FFFFFFF;
int e;
int shift;
unsigned unit;
unsigned mask;
unsigned remainder;
unsigned rounded;
unsigned half;
if (exp >= 150) {
    return uf;
}
if (exp < 126) {
    return sign;
}
if (exp == 126) {
    if (frac == 0) {
        return sign;
    }
    return sign | 0x3F800000;
}

```

```

e = exp - 127;
shift = 23 - e;
unit = 1 << shift;
mask = unit - 1;
remainder = uf & mask;
rounded = uf & ~mask;
half = unit >> 1;

```

```

    if (remainder > half ||
        (remainder == half && (rounded & unit))) {
        rounded = rounded + unit;
    }

    return rounded;
}

```

测试结果

运行 `./btest -f floatRoundEven`，结果显示错误数为 0，得分为 10/10，通过该函数的正确性测试。

17. float_i2f: 整数转换为浮点数编码

题目要求

仅使用整数运算，将有符号整数 x 转换为对应的 IEEE 754 单精度浮点数编码。无法精确表示时，采用就近舍入、平局取偶数的规则。

实现思路

首先单独处理零，然后提取符号位，并使用 `unsigned` 保存整数的绝对值，以便正确处理最小负整数。

通过循环找到绝对值中最高位的 1，设其位置为 p ，则实际指数为 p ，阶码为 $p + 127$ 。

当 $p \leq 23$ 时，所有有效位都能被精确保留，通过左移对齐并去掉隐含的最高位 1，得到尾数。

当 $p > 23$ 时，右移保留最高的 24 位，并提取被舍弃的低位。若被舍弃部分大于一半，或者恰好等于一半且保留部分为奇数，则进位。若进位导致有效位增加，则右移一位并将阶码加一。

最后，将符号位、阶码和尾数组合，得到浮点数编码。

实现代码

```

unsigned float_i2f(int x) {
    unsigned sign;
    unsigned magnitude;

```

```

unsigned exp;
unsigned frac;
unsigned remainder;
unsigned half;
int p = 31;
int shift;
if (x == 0) {
    return 0;
}
sign = x & 0x80000000;
magnitude = x;
if (sign) {
    magnitude = ~magnitude + 1;
}
while ((magnitude >> p) == 0) {
    p = p - 1;
}
exp = p + 127;
if (p <= 23) {
    frac = (magnitude << (23 - p)) & 0x7FFFFFFF;
    return sign | (exp << 23) | frac;
}
shift = p - 23;
frac = magnitude >> shift;
remainder = magnitude & ((1u << shift) - 1);
half = 1u << (shift - 1);
if (remainder > half ||
    (remainder == half && (frac & 1))) {
    frac = frac + 1;
}

```

```

    }
    if (frac == 0x1000000) {
        frac = frac >> 1;
        exp = exp + 1;
    }

    return sign | (exp << 23) | (frac & 0x7FFFFFFF);
}

```

测试结果

运行 `./btest -f float_i2f`, 错误数为 0, 得分为 10/10, 通过该函数的正确性测试。

18. bitCount: 统计二进制中 1 的数量

题目要求

统计整数 x 的 32 位二进制表示中 1 的总数。

实现思路

采用分组累加的方法。首先构造掩码 `0x55555555`、`0x33333333` 和 `0x0F0F0F0F`, 分别用于提取每两位中的低一位、每四位中的低两位和每八位中的低四位。

通过掩码、右移和加法, 依次得到每两位、每四位、每八位中 1 的数量。每次相加的结果都能保存在对应分组内, 不会向相邻分组产生进位。

随后, 将结果分别与其右移 8 位、右移 16 位后的值相加, 使最低字节保存全部 32 位的计数。由于总数范围为 $0 \sim 32$, 最后使用 `0x3F` 提取低六位, 得到答案。

实现代码

```

int bitCount(int x) {
    int mask1 = 0x55 | (0x55 << 8);
    int mask2 = 0x33 | (0x33 << 8);
    int mask4 = 0x0F | (0x0F << 8);
    mask1 = mask1 | (mask1 << 16);
    mask2 = mask2 | (mask2 << 16);
    mask4 = mask4 | (mask4 << 16);
}

```

```

    x = (x & mask1) + ((x >> 1) & mask1);
    x = (x & mask2) + ((x >> 2) & mask2);
    x = (x & mask4) + ((x >> 4) & mask4);
    x = x + (x >> 8);
    x = x + (x >> 16);

    return x & 0x3F;
}

```

测试结果

运行 `./btest -f bitCount`，错误数为 0，得分为 10/10，通过该函数的正确性测试。

19. bitReverse: 二进制位反转

题目要求

将整数 `x` 的 32 个二进制位逆序排列，即原来的第 `(i)` 位移动到第 `(31-i)` 位。

实现思路

采用逐级交换的方法，依次交换相邻的 1 位组、2 位组、4 位组、8 位组和高低两个 16 位半区，最终完成全部 32 位的反转。

每次交换时，利用掩码提取较低的一组并左移，同时将较高的一组右移并用掩码提取，最后通过按位或合并。右移后的掩码还可以清除算术右移补出的符号位。

为减少操作数，先构造 `0x0000FFFF`，再通过左移和异或依次生成其他掩码，复用中间结果。

实现代码

```

int bitReverse(int x)
{
    int mask16 = 0xFF | (0xFF << 8);
    int mask8 = mask16 ^ (mask16 << 8);
    int mask4 = mask8 ^ (mask8 << 4);
    int mask2 = mask4 ^ (mask4 << 2);
    int mask1 = mask2 ^ (mask2 << 1);
}

```

```

    x = ((x & mask1) << 1) | ((x >> 1) & mask1);
    x = ((x & mask2) << 2) | ((x >> 2) & mask2);
    x = ((x & mask4) << 4) | ((x >> 4) & mask4);
    x = ((x & mask8) << 8) | ((x >> 8) & mask8);
    return (x << 16) | ((x >> 16) & mask16);
}

```

测试结果

运行 `./btest -f bitReverse`，错误数为 0，得分为 10/10，通过该函数的正确性测试。

测试结果：

```

sibo_wang@sibo:~/Lab1$ ./check_ops.py bits.c
./btest
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 25/32 operators
isBetweenEitherOrder: 33/48 operators
mul5Sat: 17/30 operators
classifyAdd3: 21/52 operators
floatScaleThreeHalves: 36/60 operators
floatRoundEven: 23/65 operators
float_i2f: 35/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110
sibo_wang@sibo:~/Lab1$

```

在终端运行 `./check_ops.py bits.c`，19 个函数全部通过操作规则检查，操作数均未超过题目规定的上限。随后运行 `./btest`，所有函数的错误数均为 0，总得分为 110/110，通过全部正确性测试。运行结果如上图所示。

感受：

这次作业主要完成了对于二进制数的操作与计算，了解了整数和浮点数工作的原理，有些题目很难花了很长时间理解，不过在做完之后有很多收获。